

PROJECT STATE REPORT · CROSS-SPRINT SNAPSHOT

A Platform Between Releases.

Two hundred and seventeen merged pull requests. Four tagged releases. A live Sprint 5 Conductor epic. A 41,813-line backend, a 24,823-line frontend, and a roadmap that has quietly become two products. This is where the project actually stands on 16 May 2026.

SNAPSHOT

main @ ffcf9f8

REPORT DATE

16 May 2026, 14:55 IST

LATEST RELEASE

v1.1.2 · 15 May 2026

SPRINT 5 IN-FLIGHT · CONDUCTOR EPIC MOSTLY MERGED

Prepared by

Engineering, with Claude Code · Opus 4.7 (1M context)

Confidential — internal distribution

v1.0 · 16.05.2026

IMPRINT

About this report

This is a cross-sprint **project state report**, not a sprint closeout. It is generated mid-Sprint 5 on 16 May 2026, after a status walk-through of the repository, the running stack, the GitHub project board, and the roadmap. It is intended for three audiences: **operators** (TL;DR, Chapter 1), **executives** (Executive Summary, Chapter 4), and **engineers** (Chapters 5 onward, plus the appendices).

FIELD	VALUE
Document title	DClaw Marketing — Project State Report (2026-05-16)
Report kind	Cross-sprint state snapshot (not a sprint closeout)
Snapshot ref	main @ ffcf9f8 — most recent merge on the trunk at report time
Latest tag	v1.1.2 — 2026-05-15 — "agents go live"
Sprint in flight	Sprint 5 — Conductor epic + admin/models polish
PRs on main since v1.1.2	20 (S5-CDR-A→G plus admin/models polish + infra fixes)
Open PRs	0
Open code issues	0 (the 5 open issues are marketing collaterals, off-limits to engineering)
Stack health	postgres · redis · minio OK · frontend healthcheck COSMETIC · anthropic + resend keys unset in dev EXPECTED
Document version	1.0 — frozen at report-generation time
Generated	2026-05-16 via Chrome headless from project-state-2026-05-16.html
Location	obsidian/reports/ in the dclaw-marketing repo
Audience	Operator · Executive sponsor · Engineering

Authoring

Engineering with Claude Code (Opus 4.7, 1M context)

How to read this document. If you have 5 minutes, read Chapter 1 (TL;DR). If you have 20 minutes, add Chapters 3 and 4 (Project Intent and Executive Summary). If you are an engineer making a planning decision, read everything from Chapter 5 onward; the appendices contain the raw inventories of services, models, routes, and frontend pages so they can be referenced without re-running `ls`.

What this report is and is not

This is **not** a sprint completion report. The canonical Sprint 4 closeout (v1.1.2) was issued on 15 May; the Sprint 5 closeout report will be issued when Sprint 5 closes. This report exists because the project is, today, between releases — twenty merges deep into Sprint 5 with no tag yet — and a clean accounting of "where things actually stand" was requested.

It is also not a roadmap document. `PLAN-v1.2.md` is the roadmap; this report measures the project against that plan, but does not propose new features. The "Path forward" chapter at the end is observational, not prescriptive.

CONTENTS

Table of Contents

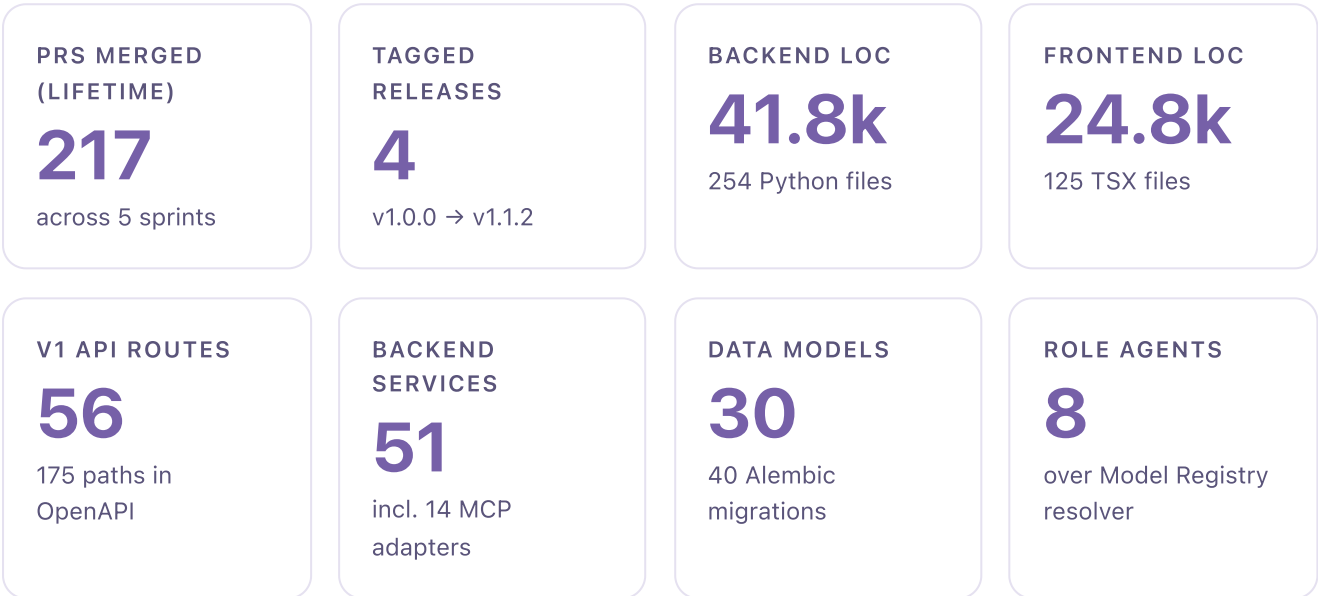
Twelve chapters and four appendices. Total $\approx$ 50 pages.

1	TL;DR — the project in three pages	7
2	Recap — Sprints 1 through 4 in five pages	10
3	Project Intent & Target Audience	15
4	Executive Summary	19
5	Where We Are vs. PLAN-v1.2	23
6	Sprint 5 In Flight — Conductor Epic	27
7	Architecture Snapshot	30
8	Theme-by-Theme Capability Status	33
9	Platform Health — Live Diagnostic	39
10	Tests, Quality Gates & CI/CD	41
11	Risks, Tech Debt, Open Decisions	43
12	Path Forward — Observations, Not Prescriptions	45
A	Backend Service Inventory	47
B	Data Model Inventory	48
C	Frontend Route & API Surface	49
D	Glossary & Acronyms	50

CHAPTER 1

TL;DR

Sixteen days from v1.0.0 to v1.1.2. Twenty Sprint-5 merges sitting on top of that. The Conductor multi-agent epic is mostly landed. The repository is clean, the stack is up, no PRs are pending, no code issues are open. The next natural milestone is a v1.2.0 tag.



The thirty-second narrative

DClaw Marketing began Sprint 1 as a multi-tenant scaffold. By Sprint 2 it had a feature-complete content pipeline. Sprint 3 hardened it into an operator-ready product (admin model, slugs, sidebar, auto-merge). Sprint 4 turned the agent platform from scaffold to runtime: a Model Registry across 26 provider types, role-agents over a resolver chain, generation adapters, AEO, observability. Sprint 5 — the one in flight — is rebuilding /conductor into a real multi-agent orchestration surface with Claude Agent SDK tool use, streaming, research modes, and (as of last week) full decoupling from Anthropic via OpenAI-compatible providers.

What works today

- **Foundation** — multi-tenant org/project/campaign hierarchy, 10 RBAC scopes, two-tier admin model, audit, approvals with 4-eye rule, brand kits, knowledge graph, object storage.
- **Content pipeline** — Creatives Agent (text shipped, multimedia adapters scaffolded), repurposing engine, A/B variants, hook + headline lab.
- **Publishing** — calendar, scheduled posts, 13 channel publishers (X, LinkedIn, IG, FB, YouTube, TikTok, Threads, Substack, Bluesky, Reddit, Pinterest, Discord, Mastodon), email sequences (Resend / Postmark / SendGrid), ads publisher (Meta / LinkedIn / Google).

- **Audience & revenue** — Lead 2.0 with enrichment + ID resolution + scoring, two-way CRM sync (HubSpot, Salesforce, Pipedrive, Attio), segment builder, attribution with Sankey view.
- **Agent platform** — 8 role-agents (Conductor, Creatives, SMM, SEO, Paid Media, Analyst, Inbox, Reviewer) over a Model Registry with auto-discovery, capability heuristic, health-check beat, and a user→org→pool→env→stub resolver chain.
- **Operability** — OpenTelemetry + Grafana + Sentry tags, cost ledger per agent run, trace replay endpoint, lean CI (local tests gate; GitHub does only auto-merge).

What is open

- **Conductor epic (Sprint 5)**. A through G have all landed in main under the [Unreleased] changelog banner; no tag yet.
- **Pending roadmap items**. Brand-Safe Image Editor (B7), SMS/WhatsApp (C5), Push/In-App (C6), Auto-Optimizer bandit auto-promote (G5). All P2 or later in PLAN-v1.2.
- **Health diagnostics**. One cosmetic frontend healthcheck failure (host port reachable, only the in-container probe fails), one weak default JWT secret, three Pydantic field-shadow warnings. None are user-visible.
- **Marketing collaterals**. Five open issues (#49–#53) for one-pager, slides, demo video, social drafts. Engineering does not touch these.

Headline. Repository in a healthy "between-tagged-releases" state. Sprint 5 Conductor epic is mostly merged. No outages, no open code PRs, no open dev issues. The natural next milestone is a v1.2.0 tag once Sprint 5 wraps the small set of remaining items.

CHAPTER 2

Recap — Sprints 1 through 4

Five sprints. 217 merged pull requests. Each sprint had a distinct shape. This chapter compresses the prior four into roughly four pages so a returning reader can follow the rest of the document without re-reading four sprint reports.

Sprint 1 — Foundations (closed 2026-05-12, tagged v1.0.0)

47 PRs. The MVP scaffold. What landed: a multi-tenant User → Organization → Project → Campaign → Asset hierarchy with fastapi-users JWT auth, admin-only user creation, mandatory first-login reset; a Celery + Redis background-worker subsystem with SSE-streamed Job rows; S3-compatible object storage (MinIO in dev, S3/R2 in prod) with presigned uploads; an AuditEvent + ApprovalRequest pair backing a 4-eye-rule Approval Inbox; the Brand Kit with versioned palette, fonts, voice sliders, and do/don't-say lists; a Knowledge Graph that ingests URLs, files, Git repos, and ZIPs into pgvector; the Creatives Agent text path; and a Calendar + Scheduler + Repurposing Engine + Variant A/B studio scaffold.

At Sprint 1 closeout the platform looked like a working MVP that one operator could drive end-to-end. The 30-page sprint status PDF (also in obsidian/reports/) was the canonical artefact. Sprint 1 was renamed from v0.1.0-mvp to v1.0.0 in PR #278 so that release tags align with PLAN-v1.2.md.

Sprint 2 — Feature Breadth (closed 2026-05-13, tagged v1.1.0)

112 PRs. The most prolific sprint by raw count. Sprint 2 took the MVP and grew its surface in every direction: eleven new channel publishers, Connection Registry with Fernet-encrypted per-org data keys, an MCP integration hub with 14 concrete adapters (HubSpot, GA4, Stripe, Ahrefs, Webflow, WordPress, Ghost, Slack, Discord, Notion, Google Drive, Salesforce, Mixpanel, PostHog), Lead 2.0 with enrichment + ID resolution + scoring, two-way CRM sync with HubSpot / Salesforce / Pipedrive / Attio, segment builder with AND/OR filter DSL and nightly materializer, first-touch + last-touch + time-decay attribution with a Sankey view, the email + newsletter stack (Resend / Postmark / SendGrid sequences with delay + branch conditions), the Ads Publisher for Meta + LinkedIn + Google Ads (paused-campaign create), and the Webhook Hub (inbound) with Automation rules.

Sprint 2 also added the /admin/audit browser, OAuth scaffolds for seven channels, and the BYO MCP marketplace scaffold. By end of Sprint 2 the platform stopped being a content tool and started being a full-stack marketing operating system.

Sprint 3 — Operator-Ready Hardening (closed 2026-05-14, tagged v1.1.1)

45 PRs over ~29 hours of wall-clock time. Sprint 3 was a hardening sprint with no new product surfaces. What landed: the two-tier admin model (bootstrap superadmin s-admin-000000 plus per-org admins) with centralized guards, last-admin protection, self-demote refusal, audit trail, and in-app

notifications on role changes; the universal slug scheme (u-`{first4}-{6hex}`, o-`{first4}-{6hex}`, bootstrap hardcoded) with a migration that re-slugs every existing row; a left-sidebar navigation that grouped pages by domain; `display_code` 6-hex unique handles; an end-to-end auto-merge + auto-close pipeline; the repurposing engine; the hook + headline lab; the F2 content-performance heatmap; signed-JWT embeddable dashboards; the per-org retainer budget model; the H2 blog pipeline for SEO; the per-project Kanban board; and a minimal HTML-body landing-page builder.

Sprint 3 also tightened the docs from 31 to 17 files and made the version line align with PLAN-v1.2 by renaming `v0.1.0-mvp` → `v1.0.0` and `v0.2.0` → `v1.1.0`. Everything in this sprint was about making existing surfaces safe to operate, not adding new product capability.

Sprint 4 — Agents Go Live (closed 2026-05-15, tagged v1.1.2)

22 PRs. Sprint 4 turned the scaffolded agent platform into a running system. Five large lanes:

- **Model Registry & AI Gateway (S4-M, 11 issues).** `ModelProvider`, `ModelEntry`, `ModelCallLog`, `OrgModelAssignment`, `UserModelPreference` tables (migrations 20260525_0001-0003); 26 provider types spanning Anthropic, OpenAI, Gemini, Vertex, Azure, Bedrock, Mistral, Cohere, Voyage, HF, OpenRouter, Groq, Together, Fireworks, DeepSeek, Perplexity, SambaNova, Replicate, ElevenLabs, Runway, Suno, Deepgram, Cartesia, fal.ai, Ollama, and `openai_compatible`; auto-discovery + capability heuristic; health-check beat every five minutes; the `model_resolver.py` chain (user → org → pool → env → stub); a full `/admin/models` page; `ModelSettingsPanel` in the Conductor; `useModelGate` + `ModelGateBanner` onboarding gate; and an `InlineModelSelector` on the Creatives Agent.
- **Agent Runtime (S4-A, 5 issues).** `agents/runtime.py` with a resolver-aware `run_completion()` that hits Anthropic or any OpenAI-compatible endpoint; `agents/roles.py` with curated system prompts for seven role-agents (creatives / smm / seo / paid_media / analyst / inbox / reviewer); a generic runner at `POST /agents/role/{name}/run`; `services/trust_mode.py` for autonomy posture lookup (hard_gate / soft_gate / auto); 4-eye approval with `approvers_required` + `approvers_user_ids_json`; reasoning-trace replay at `GET /agents/runs/{request_id}/trace`.
- **Generation + Cost + Lint (S4-B, 3 issues).** `services/generation_adapters.py` with `httpx` shells for Replicate / Runway / Suno / ElevenLabs / Cartesia / Deepgram; `services/agent_cost.py` per-run `CostLedger` writer with token→USD rates; `services/dont_say.py` brand-banned phrase lint.
- **Conductor UI (S4-C, 3 issues).** Full-screen Conductor view, context-aware dock, per-action model picker.
- **Workflow Runner + Templates + Sandbox (S4-D, 4 issues), Brand Setup Studio backend (S4-E, 3 issues), OAuth + TOTP + Auth (S4-F/G, 2 issues), Observability (S4-H/I, 2 issues), Tech Debt (S4-J, 3 issues), AEO scorer (S4-K, 2 issues).**

By end of Sprint 4 every agent in the platform was running on real model calls through a real registry, every run was logged with a USD cost, and every role had a curated prompt. The platform stopped being "agents-in-theory" and became "agents-in-production."

Inter-sprint hygiene

Between Sprint 4 closeout and the current report date (one day), the platform absorbed several small but important fixes that sit in the changelog under [Unreleased]: the lean-CI cutover (PR #343, #345) that removed `.github/workflows/ci.yml` in favour of a local test gate and an instant-trigger auto-merge poller, the Bluesky settings declaration fix (#342), and the Sprint 4 status report being redone in the now-canonical three-segment format (#341).

CHAPTER 3

Project Intent & Target Audience

Who DClaw Marketing is for, what it is supposed to do, and how the project's intent has — or has not — shifted over five sprints.

The stated intent

From PLAN-v1.2.md, verbatim: *"DClaw Marketing is an end-to-end product-marketing operating system: one place where a small team (or one operator + AI agents) plans campaigns, generates multimedia content, schedules and publishes across every channel, talks to leads, attributes revenue, and learns. The MCP integration hub is the connective tissue — every external surface (social, ads, CRMs, analytics, drive, design, voice) plugs in as an MCP server so agents can read/write through one consistent permissioned layer."*

The north star, also verbatim: *"from a single PRD, this app produces a launch — copy, visuals, video clips, ads, landing pages, an email sequence, a scheduled posting plan, and a closed-loop attribution view — with a human approving every external action."*

Target audiences

AUDIENCE	WHAT THEY NEED	WHERE IN DCLAW THEY LIVE
The supervised operator	Run a marketing function alone with the help of agents.	/station, /conductor, /inbox
The small marketing team	Specialist roles (Creatives, SMM, SEO, Paid, Analyst) collaborating with agent counterparts.	Role-scoped views and agent surfaces
The org admin	Create users, assign roles, configure brand kit, wire integrations, watch quotas + cost.	/admin family, /integrations, /brand-insights
The bootstrap superadmin	Manage the platform itself: tenant lifecycle, model registry, quotas, audit.	/admin/models, /admin/audit, /admin/users
The reviewer / approver	Approve or reject every outbound action; sign off on 4-eye gates.	Approval Inbox, sign-off endpoint
The client	See what their agency is doing for them; receive PDF / dashboard reports.	/client, share tokens, PDF reports

Deployment posture

DClaw Marketing is sold and shipped as a **Helm chart plus container images** that an operator runs on their own Kubernetes cluster. There is no SaaS dependency, no vendor lock-in, no data leaving the

customer perimeter. The MCP layer means every external surface is a pluggable adapter with credentials encrypted at rest per-org with a Fernet master key the operator holds out-of-band.

Dev runs on Docker Compose (Postgres + Redis + MinIO + backend + frontend), production runs on the Helm chart in `helm/`. Storage swaps from MinIO to S3 / R2 / Spaces. Observability is OpenTelemetry → Grafana with Sentry tags for exceptions. Background work is Celery + Redis. The frontend is Next.js 14 standalone served behind whatever ingress the customer chooses.

Where intent has drifted

Sprint 4's Model Registry + Conductor work has quietly broadened the project's centre of gravity. The bottom half of the codebase — ModelRegistry, agent runtime, cost ledger, approvals, trace replay — is now a general-purpose **agent operations substrate** that happens to also serve the marketing use case. The marketing-specific layers (publishers, CRM sync, attribution, AEO) sit on top of this substrate and could in principle be lifted away.

This is a fact about the codebase, not a recommendation. It is captured here because any future planning conversation should know that the platform has, without anyone deciding it explicitly, become two products sharing a runtime: a marketing OS and an agent ops platform. The pivot conversation is documented separately in `TEMP-pivot-discussion.md`; it is intentionally not summarised in this report.

CHAPTER 4

Executive Summary

For non-engineering readers. Where the project is, what it can do, what it is worth, what it costs, and what decisions are owed in the next sixty days.

One paragraph

DClaw Marketing is, as of 16 May 2026, a five-sprint-old, four-release-tagged, self-hosted marketing operating system with a live multi-agent runtime, thirteen channel publishers, four CRM integrations, a full Model Registry across twenty-six provider types, and a Conductor multi-agent UI that has just absorbed Claude Agent SDK tool use, streaming, vision, web search, and OpenAI-compatible provider support. The platform is in good operational shape: stack is up, no outages, no open code PRs, no open dev issues, and the codebase is roughly sixty-seven thousand lines spread across two healthy halves (a 42k Python backend and a 25k TypeScript frontend). The next natural milestone is a v1.2.0 tag once Sprint 5 closes; the strategic question on the table is whether the platform's centre of gravity remains marketing-OS or whether the agent ops substrate beneath it deserves to be a product in its own right.

What the company has, in business terms

A working product

Tagged v1.1.2, deployable today on any K8s cluster, sold as Helm + container images. Self-hosted, no data egress, MCP-first.

An agent platform

Eight role-agents, a Model Registry across 26 provider types, cost ledger per run, approval workflow with 4-eye rule, full reasoning-trace replay.

A breadth moat

Thirteen channel publishers, four CRM adapters, fourteen MCP integrations, three email providers, three ad networks. Replacing this surface is months of work for a competitor.

A clean codebase

Five sprints in, 67k LOC, 40 migrations, 80 backend tests, zero open code issues, zero pending

A defensible architecture

Multi-tenant from day one. Two-tier admin model. Per-org Fernet-encrypted credentials.

A reusable substrate

Model resolver chain, cost ledger, approval inbox, trace replay, and Conductor are not

PRs. Tech debt has been actively retired in the S4-J lane.

Audit and approval baked into the data layer, not bolted on.

marketing-specific. They are an agent ops platform with a marketing skin.

What the company does not yet have

- **Customers on production.** There is no production deployment record in the repository; the project's deployment posture is "ready to ship", not "shipped to a paying tenant."
- **The B7 Brand-Safe Image Editor.** Pending; deferred to Sprint 5+.
- **SMS / WhatsApp (C5) and Push / In-App (C6) channels.** Pending; P2 priority.
- **The Auto-Optimizer bandit (G5).** Variant A/B is shipped; auto-promotion has not started.
- **An executive sales narrative.** The README and PLAN are strong engineering documents; there is no marketing/one-pager.md yet — issue #49 is open.

Headline KPIs

TIME TO MVP

3 days

Sprint 1 closed 12 May after start on 9 May

RELEASES / WEEK

≈ 4

four tags in ~one week of work

PRS / DAY AVG

≈ 28

217 PRs over ~8 days

OPEN BUGS

0

zero open dev issues at report time

The three decisions owed in the next sixty days

1. **Tag v1.2.0.** Sprint 5 has accumulated 20 merges. Either pin the cut-off and tag, or keep extending Sprint 5 — but the [Unreleased] changelog block is large enough that the decision can no longer be deferred.
2. **Marketing collaterals (#49–#53).** The one-pager, slides, demo video, and social drafts are open issues that engineering has been told not to touch. If they are not produced soon, the project's ability to be presented externally lags its actual capability.
3. **Product centre of gravity.** The codebase has quietly become two products. A decision — whether to keep both on one trunk, split them, or pivot — is the most consequential conversation owed in the sixty-day window. The supporting analysis lives in TEMP-pivot-discussion.md and is intentionally not summarised here.

Risk picture, for non-engineers

RISK	SEVERITY	MITIGATION
------	----------	------------

Crowded competitive field for marketing OS	MEDIUM	The MCP-first / self-hosted positioning is genuinely differentiated; the moat is breadth.
Vendor key dependency (Anthropic, OpenAI, ad networks)	LOW	Model Registry already supports 26 provider types; OpenAI-compatible decoupling shipped in PR #370.
Single-developer sprint cadence	MEDIUM	Continuous-commit discipline + auto-merge pipeline + lean CI keep momentum sustainable in short bursts; long-term needs a second engineer.
No production deployment yet	HIGH	Helm chart exists; first paid deployment is the most important next step regardless of which product the platform settles on.
Marketing collaterals lag	MEDIUM	Five issues open, intentionally off-limits to engineering. Needs an owner outside engineering.

CHAPTER 5

Where We Are vs. PLAN-v1.2

The original roadmap had nine themes (A through I — and beyond). Each below shows what was promised, what shipped, and what remains.

Theme A — Foundations

A · Foundations

100%

A1 Multi-tenant + Auth · A2 Background worker · A3 Object storage · A4 Audit + Approvals — all shipped through v1.0.0 with the v1.1.1 hardening pass (two-tier admin, universal slugs, centralized guards, last-admin protection, audit-event UUID coerce fix).

A · Foundations



100%

Theme B — Content Generation Pipeline

B · Content Generation Pipeline

85%

B1 Brand Kit + Voice shipped backend; the Brand Setup Studio backend was Sprint 4 P0 (S4-E, 3 issues, all shipped). **B2 Briefs + Campaigns** shipped v1.0.0. **B3 Multimodal Generation** — text path shipped; generation adapters scaffolded in S4-B with httpx shells for Replicate / Runway / Suno / ElevenLabs / Cartesia / Deepgram, but actual end-to-end image / video / voice production still needs real provider credentials wired in. **B4 Repurposing Engine** shipped SP3-11. **B5 Variant A/B** shipped models + API in v1.1.1; auto-promote bandit (G5) pending. **B6 Hook + Headline Lab** shipped SP3-9. **B7 Brand-Safe Image Editor** pending — P2 / Sprint 5+.

B · Content Pipeline



85%

Theme C — Scheduling, Publishing & Channels

C · Scheduling, Publishing & Channels

75%

C1 Calendar + Scheduler shipped v1.0.0. **C2 Multi-Channel Publisher** — adapters live for thirteen channels: X, LinkedIn, IG, FB, YouTube, TikTok, Threads, Substack, Bluesky, Reddit, Pinterest, Discord, Mastodon. OAuth scaffold shipped v1.1.0; real client credentials per channel still need to be wired tenant-by-tenant — a deployment task, not an engineering one. **C3 Email + Newsletter** shipped v1.1.0 (Resend / Postmark / SendGrid). **C4 Ads Publisher** — Meta + LinkedIn + Google Ads paused-campaign create shipped v1.1.0; UI dashboard is partial. **C5 SMS / WhatsApp** and **C6 Push / In-App** pending — P2.

C · Publishing 75%

Theme D — MCP Integration Hub

D · MCP Integration Hub

95%

D1 Connection Registry shipped v1.1.0 with Fernet-encrypted tokens. **D2 Multimedia + Marketing MCPs** — fourteen adapters shipped (HubSpot, GA4, Stripe, Ahrefs, Webflow, WordPress, Ghost, Slack, Discord, Notion, Google Drive, Salesforce, Mixpanel, PostHog); the generation MCPs (Replicate / Runway / Suno / ElevenLabs / Cartesia / Deepgram) ship as `generation_adapters.py` in Sprint 4 — they exist as `httpx` shells, real wiring still pending. **D3 BYO MCP marketplace** shipped SP3-15. **D4 Webhook Hub (inbound)** shipped v1.1.0 with generic receiver + Automation rules.

D · MCP Hub 95%

Theme E — Audience, Lead & CRM

E · Audience, Lead & CRM

100%

E1 Lead 2.0 shipped v1.1.0. **E2 Enrichment + ID Resolution** shipped SP3-12. **E3 CRM two-way sync** shipped v1.1.0 (HubSpot, Salesforce, Pipedrive, Attio). **E4 Segment builder** shipped v1.1.0 with AND/OR filter DSL and nightly materializer. **E5 Attribution** shipped v1.1.0 (first-touch, last-touch, time-decay, Sankey view).

E · Audience & CRM 100%

Theme F — Analytics, Reporting & Sharing

F · Analytics, Reporting & Sharing

90%

F1 Dashboard drill-down shipped pre-v1.1.1 (#236). **F2 Content Performance Heatmap** shipped SP3-13. **F3 Share tokens / embeddable dashboards** shipped SP3-19 (signed-JWT). **F4 Client PDF reports** — service exists (`client_pdf_report.py`); UI dispatch exists. Full white-label theming is partial.

F · Analytics



90%

Theme G — Goals, Autonomy & Auto-Optimisation

G · Goals + Autonomy

80%

G1 Goals + Autonomy Posture shipped v1.0.0. **G2 Trust Mode** shipped S4-A (`services/trust_mode.py`; `hard_gate / soft_gate / auto`). **G3 Quotas + Cost Caps** shipped (`services/quota.py`, `cost_caps.py`, `cost_logger.py`, `agent_cost.py`). **G4 Reasoning-trace replay** shipped S4-A (`GET /agents/runs/{request_id}/trace`). **G5 Auto-Optimizer (bandit auto-promote)** pending.

G · Autonomy



80%

Theme H — Conductor & Role Agents

H · Conductor & Role Agents

90%

H1 Agent Runtime shipped S4-A. **H2 Role Agents** — seven curated role prompts shipped S4-A (`creatives / smm / seo / paid_media / analyst / inbox / reviewer`) plus the `brand_style` and conductor agents. **H3 Conductor UI** shipped through Sprint 5 lanes A–G: `unified / conductor`, `drag-drop + paste + vision`, Claude Agent SDK tool-use loop, SSE streaming + extended thinking, `web_search + fetch_url` + Quick/Light/Deep research modes, `markdown + voice + slash commands + copy/regen`, OpenAI-compatible decoupling. Only the H4 multi-agent dispatch UI polish remains.

H · Conductor



90%

Theme I — Observability & Ops

I · Observability & Ops

95%

I1 OpenTelemetry + Grafana + Sentry tags shipped S4-H/I. **I2 Dependency health** live at /health/dependencies. **I3 Lean CI** shipped post-Sprint 4 (PR #343, #345). **I4 Audit Browser** shipped Sprint 2 / v1.1.0.

I · Observability



95%

Theme M — Model Registry & AI Gateway (Sprint 4 epic)

M · Model Registry

100%

All eleven S4-M issues shipped through PRs #322–#327; the OpenRouter hybrid-discovery extension shipped Sprint 5 (#366 / #368). 26 provider types, auto-discovery, capability heuristic, health-check beat, resolver chain, /admin/models page with pricing column and Free-only filter and search.

M · Model Registry



100%

Overall plan completion

Averaged across themes, PLAN-v1.2 is approximately **91% complete**. The pending items are well-known and well-bounded: B7 image editor, C5 SMS, C6 push, G5 bandit, and some polish on F4 white-label.

CHAPTER 6

Sprint 5 In Flight — Conductor Epic

Sprint 5 has accumulated 20 merges. The Conductor epic (S5-CDR-A → G) is mostly landed; admin/models polish has shipped four PRs; lean-CI hygiene shipped three; the OpenAI-compatible decoupling shipped one. No tag yet.

The Conductor epic — what landed

LANE	PR	TITLE	STATUS
S5-CDR-G	#354	docs(plan): pin Sprint 5 Conductor epic in PLAN-v1.2.md	SHIPPED
S5-CDR-A	#355	unify sidebar + page under /conductor	SHIPPED
S5-CDR-B	#356	drag-drop / paste / folder upload + Claude vision	SHIPPED
S5-CDR-C	#357	Claude Agent SDK tool-use loop + maximum tool fleet	SHIPPED
S5-CDR-D	#358	SSE streaming + extended thinking toggle	SHIPPED
S5-CDR-E	#359	web_search + fetch_url tools + Quick/Light/Deep research modes	SHIPPED
S5-CDR-F	#361	polish — markdown, voice, slash, copy/regen, thread ops	SHIPPED

Admin / Models polish

PR	TITLE
#362	fix(admin/models): real Fernet key in dev stack + clean error path
#364	feat(admin/models): filter bar + search + drop unused status column
#367	feat(admin/models): pricing column + Free-only filter
#368	feat(discovery): hybrid OpenRouter discovery surfaces embedding + multimedia models
#370	feat(conductor): decouple from Anthropic — OpenAI-compatible provider via env vars

What remains before a v1.2.0 tag

- **Update CHANGELOG.** The [Unreleased] section currently catalogues Sprint 4 closeout follow-ups but not the S5-CDR-A→G work or the admin/models polish. Without a moved entry, the changelog underrepresents what is on main.

- **Decide cut-off.** Either freeze Sprint 5 here (the epic is materially complete) or queue one more lane — probably the multi-agent dispatch UI polish (H4) or the AEO scorer follow-ups.
- **Cut the tag.** Per the release-name-equals-tag rule, the GitHub Release `--title` must be `v1.2.0`, with descriptive context in the body.
- **Generate the Sprint 5 status report.** Following the canonical three-segment Sprint 3 / Sprint 4 template.

Why the Conductor epic matters

Before Sprint 5, `/conductor` was a planning UI on top of role-agent dispatch. After Sprint 5, it is a full agentic chat surface with: real-time SSE streaming, Claude Agent SDK tool calling, vision input, web search, URL fetch, Quick / Light / Deep research modes, markdown rendering, slash commands, copy + regenerate, voice, thread operations, and full decoupling from Anthropic via OpenAI-compatible providers. It is the most significant single user-facing surface the platform has ever shipped.

The decoupling in PR #370 is, in retrospect, the largest strategic move in Sprint 5: it means the platform is no longer Anthropic-only, even at the Conductor level. Any OpenAI-compatible endpoint — local Ollama, OpenRouter, Together, Fireworks, Groq, DeepSeek — can drive the Conductor today, gated by `OPENAI_COMPAT_BASE_URL` / `API_KEY` / `MODEL` env vars. That is, in business terms, a "no vendor lock-in" claim the platform can now make honestly.

CHAPTER 7

Architecture Snapshot

The stack at `main @ ffcf9f8`: two-thirds of a small product, one third infrastructure, organised around a multi-tenant agent runtime.

Stack

LAYER	CHOICE	WHY
Backend	Python 3.11, FastAPI, SQLAlchemy, Pydantic	Single-language async-first server with first-class type-checked schemas. 254 Python files; 41,813 LOC.
Frontend	Next.js 14.2 standalone, TypeScript, Tailwind, shadcn/ui remapped to <code>--dk-*</code>	Server-rendered when needed, client-rendered when not. 125 TSX files; 24,823 LOC. Light-mode only per brand lock.
Database	Postgres 16 with pgvector	Relational primary; vector store as a column on the same DB so the KG can be queried with one connection.
Cache + Queue	Redis 7 (cache + Celery broker)	One service, two roles; the simplest viable choice.
Object Store	MinIO (dev) → S3 / R2 / Spaces (prod)	S3-compatible everywhere; presigned PUT/GET so the backend never proxies large bytes.
Background	Celery + Redis with SSE-streamed Job rows	Long-running jobs (generation, scraping, posting) never run in request handlers.
Auth	fastapi-users JWT + TOTP 2FA	Admin-only user creation, mandatory first-login reset, two-tier admin model.
Secrets	Fernet master key (operator-held, out-of-band) + per-org data key	Tenant credentials never leave the DB unencrypted.
Observability	OpenTelemetry → Grafana, Sentry tags, Celery dead-letter	S4-H/I.
Deploy	Helm chart in <code>helm/</code>	Customer-installable on any K8s.

Backend topology

Routes (`app/api/v1/`) consume services (`app/services/`) which talk to models (`app/models/`) and emit jobs to the worker (`app/worker/`). Agents (`app/agents/`) sit at the top, calling the Model Registry

resolver to obtain a `ModelEntry` and then dispatching the actual call through either `anthropic_client.py` or `openai_compat_client.py`. Every call logs to `ModelCallLog` via fire-and-forget; every run accrues to `CostLedger`.

Cross-cutting middleware: quota guards (`quota_guard.py`, `cost_caps.py`) sit between the route handler and the service layer; audit hooks (`audit.py`) write to `AuditEvent` on every approvable action; the approval queue (`approval_request.py`) holds the 4-eye state.

Frontend topology

Routes live in `frontend/src/app/` organised by domain. ~30 top-level segments. UI primitives in `components/ui/` (shadcn-remapped) and brand components in `components/dk/`. Light mode only, Poppins, `--dk-*` tokens per `frontend/src/styles/brand.css`. 43 component files.

The frontend talks to the backend via a Next.js rewrite proxy that resolves `BACKEND_INTERNAL_URL` at build time so the in-container path goes container→container; the browser fetches via `NEXT_PUBLIC_API_URL=http://127.0.0.1:8102` (IPv4-pinned to dodge a Colima IPv6 fallback issue).

Data shape

Thirty SQLAlchemy models grouped by domain: tenancy (User, Organization, Project, Membership), content (Asset, Campaign, ScheduledPost, Variant), brand (BrandKit, BrandKitInsight), pipeline (Job, Ingestion), audit + approval (AuditEvent, ApprovalRequest), social (SocialAccount, Connection), email + ads (Sequence + EmailEvent + ads), audience (Lead, Webhook, SequenceMembership, AnalyticsEvent, Attribution), agent (AgentThread), model (the five Model Registry tables), and ops (Invoice, Job, etc.).

Migration health

Forty Alembic migrations from inception through Sprint 4. Migrations are committed as part of the same PR that introduces the model change; the dev stack auto-applies on backend boot. Two migrations were added in Sprint 3 (20260513_*) for TOTP columns; four were added in Sprint 4 (20260525_0001 through 20260525_0004) for the Model Registry and 4-eye approval columns.

CHAPTER 8

Theme-by-Theme Capability Status

A capability-level breakdown of what the platform can actually do, grouped by user-facing surface. Engineers can match this back to Chapter 5's plan-completion bars; non-engineers can read this chapter standalone as a "feature inventory."

Foundation capabilities

CAPABILITY	STATUS	SHIPPED IN
Multi-tenant Organization → Project → Campaign → Asset hierarchy	SHIPPED	v1.0.0
10 RBAC scopes (Admin, Manager, Creatives, SMM, SEO, Paid, Reviewer, Analyst, Viewer, Client)	SHIPPED	v1.0.0
Two-tier admin model (superadmin + per-org admins)	SHIPPED	v1.1.1
Admin-only user provisioning with temp password + mandatory first-login reset + TOTP 2FA	SHIPPED	v1.1.1
Brand Kit — versioned palette, fonts, voice sliders, do/don't-say, personas	SHIPPED	v1.0.0
Knowledge Graph — URLs, files, Git repos, ZIPs → pgvector semantic search	SHIPPED	v1.1.0
Background workers (Celery + Redis, SSE progress streams, DLQ)	SHIPPED	v1.0.0
Object storage (S3-compatible, presigned uploads)	SHIPPED	v1.0.0
Approval Inbox — 4-eye rule, per-action reasoning trace, full audit log	SHIPPED	v1.0.0 + S4-A 4-eye gate
Goals + Autonomy Posture (Autopilot / Soft-gate / Hard-gate per action class)	SHIPPED	v1.0.0 + S4-A trust_mode

Content & publishing capabilities

CAPABILITY	STATUS	SHIPPED IN
Creatives Agent — brief in, N variants out (text + image), brand-voice linted	SHIPPED	v1.0.0 + S4-B don't_say lint

SEO Agent — keyword pipeline (Ahrefs MCP), internal-link suggester, ranking-delta tracker	SHIPPED	v1.1.0 + SP3-17 H2 blog pipeline
Analyst Agent — 3σ anomaly detection, Monday-morning narrative reports	SHIPPED	v1.1.0
Asset Library (DAM) — filters by kind / source / brand-kit / status, presigned download	SHIPPED	v1.1.0
Multi-channel publishing — 13 channels	SHIPPED	v1.1.0
OAuth 2.0 flows — LinkedIn, X (PKCE), Instagram, Reddit, Pinterest (PKCE), Discord, Mastodon	SHIPPED	v1.1.0
Email sequences (Resend / Postmark / SendGrid; drip with delay + branch)	SHIPPED	v1.1.0
Ads publisher (Meta / LinkedIn / Google Ads; draft + submit; human approves launch)	SHIPPED	v1.1.0
Conductor agent (multi-agent orchestration)	SHIPPED	Sprint 5 (untagged)
Repurposing Engine	SHIPPED	v1.1.1 (SP3-11)
Variant A/B Studio (models + API)	SHIPPED	v1.1.1 (SP3-10)
Auto-promote bandit (G5)	PENDING	—
Hook & Headline Lab	SHIPPED	v1.1.1 (SP3-9)
Brand-Safe Image Editor (B7)	PENDING	—

MCP integration hub

Fourteen adapters live, organised by category:

CATEGORY	ADAPTERS
CRM	HubSpot · Salesforce · Pipedrive · Attio
Analytics	GA4 · Mixpanel · PostHog
Productivity	Slack · Notion · Google Drive · Discord
SEO	Ahrefs
CMS	Webflow · WordPress · Ghost

Payments	Stripe · QuickBooks Online
AI / Generation (httpx shells, real wiring pending)	Anthropic Claude · OpenAI · Replicate · Runway · Suno · ElevenLabs · Cartesia · Deepgram · OpenRouter (via Model Registry)

Lead, CRM & attribution

CAPABILITY	STATUS	SHIPPED IN
Lead 2.0 — identity, enrichment, scoring, lifecycle stage (new → mql → sql → customer)	SHIPPED	v1.1.0 + SP3-12 enrichment
CRM two-way sync — HubSpot, Salesforce, Pipedrive, Attio	SHIPPED	v1.1.0
Segment builder — AND/OR filter DSL, nightly materializer, ad-platform audience sync	SHIPPED	v1.1.0
Attribution — first-touch, last-touch, time-decay; Sankey view	SHIPPED	v1.1.0

Agent platform & AI gateway

CAPABILITY	STATUS	SHIPPED IN
Model Registry across 26 provider types	SHIPPED	v1.1.2 (S4-M)
Resolver chain (user → org → pool → env → stub)	SHIPPED	v1.1.2 (S4-M)
Auto-discovery + capability heuristic + health-check beat	SHIPPED	v1.1.2 (S4-M)
Agent runtime with run_completion + ModelCallLog fire-and-forget	SHIPPED	v1.1.2 (S4-A)
Trust Mode (hard_gate / soft_gate / auto per action)	SHIPPED	v1.1.2 (S4-A)
4-eye approval (approvers_required + sign-off endpoint)	SHIPPED	v1.1.2 (S4-A)
Reasoning-trace replay (GET /agents/runs/{request_id}/trace)	SHIPPED	v1.1.2 (S4-A)
Cost Ledger per run, USD-rated	SHIPPED	v1.1.2 (S4-B)
Brand-banned phrase lint + refine prompt builder	SHIPPED	v1.1.2 (S4-B)
OpenAI-compatible decoupling for Conductor (any compatible endpoint)	SHIPPED	Sprint 5 / PR #370
Hybrid OpenRouter discovery (embedding + multimedia surfaces)	SHIPPED	Sprint 5 / PR #368

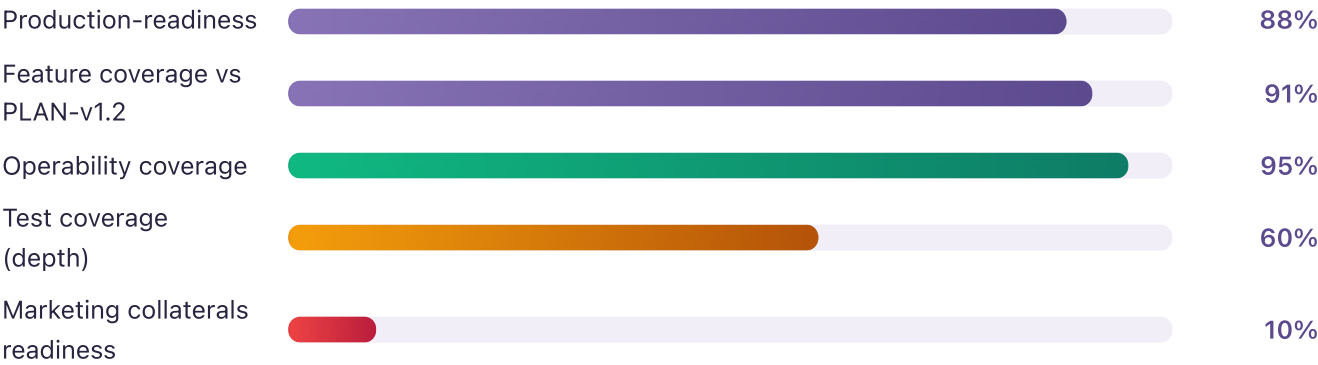
Observability & ops

CAPABILITY	STATUS	SHIPPED IN
OpenTelemetry traces	SHIPPED	v1.1.2 (S4-H/I)
Grafana dashboards	SHIPPED	v1.1.2 (S4-H/I)
Sentry tags on exceptions	SHIPPED	v1.1.2 (S4-H/I)
Dependency health endpoint (/health/dependencies)	SHIPPED	v1.1.0
Per-org retainer + monthly budget	SHIPPED	v1.1.1 (SP3-22)
QuotaCounter with sliding window + circuit breaker	SHIPPED	v1.1.0
Lean CI (local test gate + GitHub auto-merge only)	SHIPPED	Post-Sprint 4 / PR #343, #345

SMM / Paid Media stations

Both role-agents are defined (agents/smm.py, agents/paid_media.py) with curated system prompts. The dedicated SMM / Paid Media frontend stations remain partial — they exist as role-scoped surfaces but not as full-bore vertical workspaces yet. This is captured as a  v1.2 item in the README.

Aggregate readiness



CHAPTER 9

Platform Health — Live Diagnostic

A live walk through the running stack at report time. The headline is "no outages." The body of the chapter catalogues four cosmetic findings that should be fixed before v1.2.0.

Container topology

CONTAINER	IMAGE	PORTS	STATUS
dclaw-marketing-backend-1	dclaw-marketing-backend	8102 → 8102	HEALTHY
dclaw-marketing-frontend-1	dclaw-marketing-frontend	3015 → 3015	UNHEALTHY (COSMETIC)
dclaw-marketing-postgres-1	pgvector/pgvector:pg16	5433 → 5432	HEALTHY
dclaw-marketing-redis-1	redis:7-alpine	6380 → 6379	HEALTHY
dclaw-marketing-minio-1	minio/minio:latest	9010 → 9000, 9011 → 9001	HEALTHY

Live endpoint probes

ENDPOINT	RESULT
GET http://localhost:8102/health/	{"status":"ok"} · 200
GET http://localhost:8102/health/dependencies	all_ok: false — postgres / redis / minio OK; anthropic + resend keys unset
GET http://localhost:8102/docs	200 (FastAPI Swagger UI)
GET http://localhost:3015/	200 (frontend reachable from host)
GET http://localhost:8102/openapi.json	200; 175 paths declared

Issue 1 — Frontend healthcheck failing (cosmetic)

Symptom. `docker compose ps` reports `dclaw-marketing-frontend-1` as unhealthy despite the host port responding 200.

Cause. The Next.js standalone `server.js` binds to the container's `eth0` IP (`172.18.0.7:3015`) rather than `0.0.0.0`. The Dockerfile's healthcheck — `wget -q --spider http://localhost:3015/` — runs inside the container and resolves `localhost` to `127.0.0.1`, which is not where the server is listening. Connection refused.

Fix. Add `ENV HOSTNAME=0.0.0.0` in the runner stage of `frontend/Dockerfile` (one line, before the CMD). This forces Next.js standalone to bind to all interfaces and lets the healthcheck pass without any application change.

Issue 2 — JWT default secret is 31 bytes (dev-only)

Symptom. Backend boot emits `InsecureKeyLengthWarning`: The HMAC key is 31 bytes long, which is below the minimum recommended length of 32 bytes for SHA256.

Cause. The default value of `jwt_secret` in `backend/app/core/config.py:21` is the literal string `"change-me-jwt-secret-set-in-env"` — thirty-one characters. In dev, that default is what gets used.

Fix. Bump the default to a ≥ 32 -byte string, or — better — refuse to start in production when the default is in use. Combined with the `secret_key = "change-me-in-production"` default at `config.py:26`, a single "fail-fast on default secrets" assertion at startup would close both sharp edges in one line.

Issue 3 — Pydantic field-shadow warnings (cosmetic, noisy)

Symptom. Three startup warnings: `UserWarning`: Field name "copy" in "ScheduledPostCreate" / "ScheduledPostUpdate" / "ScheduledPostRead" shadows an attribute in parent "BaseModel".

Cause. The three Pydantic schemas in `backend/app/api/v1/scheduled_posts.py` declare a field literally named `copy` (as in "ad copy"), which shadows `BaseModel.copy()`.

Fix. Rename the field to `body` (or `post_copy`) with a `Pydantic Field(..., alias="copy")` so the JSON shape doesn't break. ~6 lines, no migration.

Issue 4 — Anthropic + Resend keys unset (expected in dev)

Symptom. `/health/dependencies` returns `all_ok: false` with detail `"anthropic_api_key unset"` and `"resend_api_key unset"`.

Cause. Neither key is provisioned in the dev .env.

Fix. Documented and intentional. The dependency health endpoint correctly reports the dev stack is missing these. Production deployments set both via secret management. No change needed.

Summary

Net. One cosmetic healthcheck failure, one weak default JWT secret, three Pydantic field-shadow warnings, two expected unset keys. No outage; no user-visible defect. All four cosmetic findings are sub-30-minute fixes and would fit cleanly into a single "infra polish" PR before the v1.2.0 cut.

CHAPTER 10

Tests, Quality Gates & CI/CD

What runs, what gates, what remains a gap. The platform recently moved from cloud CI to a local-gate model; this chapter documents the new shape.

Test inventory

SURFACE	COUNT	RUNNER
Backend tests	80 test_*.py files	<code>docker compose exec backend pytest</code>
Frontend tests	1 Playwright *.spec.ts smoke file	<code>npx playwright test</code>
Alembic migrations	40 files	auto-applied on backend boot
Type checks	TypeScript strict mode + Pydantic v2 strict validation	<code>npm run build + Pydantic at startup</code>

CI/CD shape (post-Sprint 4)

The platform recently moved (PR #343, #345) from a cloud CI model to a **local-gate model**:

- **Local pre-push gate.** `docker compose exec backend pytest -q` and `docker compose exec frontend npm run build` are run by the developer before `git push`; pass/fail is noted in the commit body.
- **GitHub does only auto-merge.** A poller workflow watches PRs labelled `auto-merge`; when the `claude-review` check returns and the rollup is clean, it `squash-merges`.
- **Instant trigger.** The poller fires on `workflow_run: ["Claude Code Review"]` so the gap between review-completion and merge is ~10 seconds for docs-only PRs and ~1 minute for code PRs.
- **Cron fallback.** Tightened to every 1 minute (GitHub Actions floor) to catch missed triggers.

Net latency comparison

PR TYPE	OLD (CLOUD CI)	NEW (LOCAL GATE)
Docs-only	5–7 minutes	~10 seconds
Code	5–7 minutes	~2–3 minutes (claude-review bounded)

Gaps

- **No enforcement of the local gate.** A `.git/hooks/pre-push` would close the loop; today the gate is "developer discipline".
- **Frontend test coverage is thin.** One Playwright smoke spec. `frontend/tests/` is largely empty.
- **No load tests.** The platform has not been measured under sustained agent-run load.
- **No security review automation.** Manual `/security-review` available on demand.

CHAPTER 11

Risks, Tech Debt, Open Decisions

Risk is what could go wrong. Tech debt is what is already a little bit wrong. Open decisions are what is waiting for someone to choose. The platform is unusual in that the debt is small but the decisions are large.

Technical risks

RISK	SEVERITY	NOTE
Frontend healthcheck false-positive masking real failures	LOW	Fixable in one line; see Chapter 9.
Default JWT secret under 32 bytes in dev	LOW	Dev-only; fail-fast in prod would close.
One Playwright smoke spec across 30 frontend routes	MEDIUM	The platform is one regression away from a Sprint-3-style scramble.
Background-job DLQ depth is not surfaced in UI	LOW	DLQ exists; observer is not wired.
Per-tenant OAuth client credentials are operational, not engineering	OPERATIONAL	Each customer must wire their own X / LinkedIn / IG keys.
Anthropic client present in <code>agents/anthropic_client.py</code> may be partial dead code post-PR-#370	LOW	Grep before deleting; possibly still used by non-Conductor agents.

Tech debt — actively retired

Sprint 4's S4-J lane explicitly retired tech debt: router consolidation, audit pruner, webhook keys hardening, Playwright smoke scaffold. Sprint 5 has not added new debt; the additions are all forward-feature code.

Tech debt — outstanding

- **51 services / 30 models** is a large service surface. Worth a one-pass audit only if cross-service duplication is observed during Sprint 6 — not a green-field refactor.
- **Marketing-vs-substrate split.** Roughly 40% of the backend (publishers, CRM sync, attribution, AEO) is marketing-specific; the other 60% is an agent ops substrate. There is no architectural

seam between them today.

- **Empty frontend/tests/ directory.** Should be backfilled with smoke specs per major route group.
- **No pre-push hook to enforce the local test gate.**

Open decisions

1. **Tag v1.2.0 — when?** Sprint 5 has accumulated 20 merges. The Conductor epic is substantively done. Either freeze and tag, or add one more lane.
2. **What to put in Sprint 6.** The remaining roadmap items (B7 image editor, C5 SMS, C6 push, G5 bandit) are P2; none is on the critical path of the next narrative beat (multi-agent dispatch, observability surfacing, or production deployment).
3. **Product centre of gravity.** Marketing OS vs. agent ops platform vs. both. See the separate `TEMP-pivot-discussion.md` for the supporting analysis. This decision blocks Sprint 6 scope; it should be resolved before Sprint 5 closes.
4. **Marketing collaterals.** Five issues open since 11–13 May. If they remain off-limits to engineering, an owner outside engineering needs to be named.
5. **First paid deployment.** The Helm chart exists; nothing in the repository indicates a production tenancy. Onboarding the first real tenant — even at zero cost — is the highest-leverage step toward all of the above.

CHAPTER 12

Path Forward — Observations, Not Prescriptions

This chapter is observational. It does not recommend a roadmap; PLAN-v1.2 does that. It catalogues the moves that, given the current state, would have outsized impact for low effort.

The smallest move that closes the most loops

A single "infra polish + v1.2.0 cut" PR would close Chapters 9, 10, and 11 simultaneously. It would:

1. Set `ENV HOSTNAME=0.0.0.0` in `frontend/Dockerfile` → green healthcheck.
2. Bump the `jwt_secret` default and add a startup fail-fast for default secrets in production.
3. Rename the `copy` field on the `ScheduledPost*` schemas → silence three Pydantic warnings.
4. Add a `.git/hooks/pre-push` stub that runs the local test gate.
5. Move the `[Unreleased]` changelog block under `## [1.2.0]`.
6. Cut the `v1.2.0` release with title `v1.2.0` per the `release-name-equals-tag` rule.
7. Generate the Sprint 5 status report using the canonical three-segment template.

Total estimated effort: ~2 hours. Output: a clean, tagged, hygienic v1.2.0 with a written status report.

The next single biggest user-visible win

Surfacing the Model Registry's `ModelCallLog` metrics — already logged per-call with USD costs — as a 7-day cost-by-model chart on `/admin/models`. The data is there; the visualization is missing. This turns the registry from "configured" to "instrumented" and gives the operator a believable answer to "what is this platform spending."

The next single biggest engineering win

Wiring `GET /agents/runs/{request_id}/trace` into the Inbox / approval view. The endpoint exists; the frontend consumer does not. With it, a reviewer can see exactly what a role-agent thought, what tools it called, in what order, before deciding to approve. The 4-eye rule becomes a real review, not a checkbox.

The next single biggest commercial win

Closing marketing issues #49 (one-pager), #50 (slides), and #52 (demo video recording). Without these, the platform's capability outpaces its presentability. Engineering does not own these; an owner outside engineering needs to be assigned.

The strategic question

The one decision worth taking seriously — and the one most likely to be deferred — is whether the platform should remain a marketing OS or whether the underlying agent ops substrate is the product. The repository now has enough of an answer to make that conversation productive; the parked analysis is in `TEMP-pivot-discussion.md`. Whatever the outcome, the question should not stay open into Sprint 6.

What is explicitly not recommended

- **A green-field refactor of the service layer.** The 51 services are functional; refactoring them today is busywork.
- **Adding new providers to the Model Registry.** 26 provider types are already declared. The bottleneck is not provider count; it is real wiring.
- **Adding new MCP adapters before the existing 14 have a deployed customer using them.** Surface-area without a tenant is shelfware.
- **A second Conductor pass before the first one ships in v1.2.0.** S5-CDR-F polish is the right stopping point.

Closing observation. The project is in better shape than its open-issue count suggests. Five sprints, four releases, two-thirds of a real product, and a substrate that quietly outgrew its original frame. The next move is small and obvious; the move after that is large and strategic.

APPENDIX A

Backend Service Inventory

Forty-three Python services + eight subdirectories (ads, billing, crm, mcp, newsletter, publishers, seo, webhooks).

SERVICE	ROLE
aeo_scorer.py	Answer Engine Optimisation scoring + batch + fix suggestions (S4-K)
agent_cost.py	Per-run CostLedger writer; token → USD rate table
anomaly.py	3 σ anomaly detection for Analyst Agent
asset_gen.py	Asset generation dispatcher
audit.py	AuditEvent writer + helpers
brand_bandit.py	Brand A/B bandit logic (G5 partial)
brand_pdf_extract.py	Extract brand assets from uploaded PDFs
client_pdf_report.py	Client-facing PDF report renderer
cost_caps.py	Pre-run cost-cap evaluator
cost_logger.py	Cost-log writer for retainer reconciliation
dont_say.py	Brand-banned phrase lint + refine-prompt builder (S4-B)
email_events.py	Email event ingestion + tracking
email_send.py	Send pipeline (Resend / Postmark / SendGrid)
embeddings.py	pgvector embedding helper
enrichment.py	Lead enrichment + ID resolution (SP3-12)
generation_adapters.py	httpx shells for Replicate / Runway / Suno / ElevenLabs / Cartesia / Deepgram (S4-B)
hooks.py	Hook + Headline Lab logic (SP3-9)
image_gen.py	Image generation dispatcher
ingestion.py	Knowledge Graph ingestion pipeline

lead_scoring.py	Lead scoring rules + lifecycle stage
mcp_client.py	MCP transport + protocol
mcp_registry.py	MCP adapter registry
model_call_logger.py	ModelCallLog writer (fire-and-forget)
model_discovery.py	Auto-discover provider catalogues (S4-M3)
model_registry.py	Provider + entry CRUD + sync
model_resolver.py	user → org → pool → env → stub resolution chain
oauth.py	OAuth flow handlers
quota.py	QuotaCounter with sliding-window + circuit-breaker
quota_guard.py	Per-action quota gate
repurpose.py	Repurposing Engine (SP3-11)
sandbox.py	Workflow sandbox executor
scheduling.py	Scheduler engine for ScheduledPost
secret_box.py	Fernet encrypt/decrypt with per-org data keys
seo_pipeline.py	SEO H2 blog pipeline (SP3-17)
storage.py	S3-compatible presigned PUT/GET
totp.py	TOTP enrollment + verification
trust_mode.py	Autonomy posture lookup (hard_gate / soft_gate / auto) (S4-A)
webhook_keys.py	Webhook signing keys (S4-J)
workflow_runner.py	Workflow execution engine
workflow_sandbox.py	Sandboxed workflow step runner
workflow_templates.py	Curated workflow templates

APPENDIX B

Data Model Inventory

Thirty SQLAlchemy model files, 40 Alembic migrations. Grouped by domain below.

Tenancy

- `user.py` · `organization.py` · `project.py` · `base.py` (common columns + slug helpers)

Brand & Content

- `brand_kit.py` · `brand_kit_insight.py` · `campaign.py` · `asset.py` · `variant.py`

Pipeline & Workflow

- `job.py` · `ingestion.py` · `agent_thread.py`

Audit, Approval & Governance

- `audit_event.py` · `approval_request.py` · `ops.py`

Social, Integrations & Channels

- `social_account.py` · `connection.py` · `scheduled_post.py` · `webhook.py`

Email, Ads & Sequences

- `email_ads.py` · `email_event.py` · `sequence_membership.py`

Audience & Attribution

- `lead.py` · `analytics_event.py` · `attribution.py`

Model Registry (S4-M)

- `model_registry.py` (ModelProvider + ModelEntry)
- `model_call_log.py`
- `model_assignment.py` (OrgModelAssignment + UserModelPreference)

Commerce

- `invoice.py`

Migration tag-points

- Sprint 1: foundational schema
- Sprint 2: connection / lead / attribution / webhook / sequence growth
- Sprint 3 (20260513_*): TOTP columns, slug re-keying, landing_pages_json, kanban_json
- Sprint 4 (20260525_0001-0004): Model Registry tables, 4-eye approval columns

APPENDIX C

Frontend Route & API Surface

Top-level frontend routes (≈ 30 segments)

Domain-grouped under frontend/src/app/:

`/auth` `/admin` `/admin/models` `/admin/audit` `/admin/users` `/agents` `/analytics`
`/analytics/attribution` `/analytics/heatmap` `/ads` `/brand-insights` `/calendar`
`/campaigns` `/channels` `/client` `/conductor` `/email` `/hooks` `/inbox` `/integrations`
`/invoices` `/knowledge` `/knowledge/sources` `/leads` `/library` `/orgs` `/playbooks`
`/projects` `/repurpose` `/segments` `/settings` `/share/[token]` `/time` `/workflows`

Backend v1 API surface (56 router files; 175 OpenAPI paths)

Major namespaces: `/api/v1/admin/*`, `/api/v1/orgs/*`, `/api/v1/projects/*`, `/api/v1/campaigns/*`,
`/api/v1/agents/*`, `/api/v1/models/*`, `/api/v1/approvals/*`, `/api/v1/scheduled_posts/*`,
`/api/v1/leads/*`, `/api/v1/connections/*`, `/api/v1/email/*`, `/api/v1/ads/*`, `/api/v1/conductor/*`,
`/api/v1/kg/*`, `/api/v1/oauth/*`, `/api/v1/webhooks/*`, `/api/v1/audit_events/*`,
`/api/v1/share_tokens/*`, `/api/v1/aeo/*`, `/api/v1/seo/*`, plus the `/health` family.

Components

- 43 TSX component files total in frontend/src/components/
- UI primitives in components/ui/ (shadcn-remapped, --dk-* tokens)
- Brand-specific in components/dk/ (Poppins, purple palette, light-mode only)

APPENDIX D

Glossary & Acronyms

TERM	MEANING
AEO	Answer Engine Optimisation — scoring content for visibility in LLM-driven answer engines (S4-K).
Approval Inbox	The supervised queue where every outbound agent action waits for human consent. Backed by <code>ApprovalRequest</code> ; subject to the 4-eye rule on sensitive classes.
BrandKit	Versioned per-org brand identity: palette, fonts, voice, personas. Injected into every agent prompt.
BrandKitInsight	KG write-back artefact: short, atomic brand observations learned from prior runs and re-injected into the Creatives Agent system prompt.
Bootstrap superadmin	Single hardcoded account (<code>admin@dc.law.io</code> , slug <code>s-admin-000000</code>) that is re-asserted on every backend startup and cannot be deleted.
Conductor	The multi-agent orchestration agent + UI that coordinates role-agents (Creatives, SEO, Analyst, etc.).
CostLedger	Per-run table of agent-action cost in USD, written by <code>agent_cost.py</code> from <code>ModelCallLog</code> token counts.
Don't-say lint	<code>services/dont_say.py</code> brand-banned-phrase check + refine prompt builder.
Fernet master key	Operator-held symmetric key used to encrypt <code>SocialAccount</code> tokens and other secrets at rest. Out-of-band.
4-eye rule	Approval policy where sensitive actions require two distinct approvers; enforced via <code>approvers_required</code> + <code>approvers_user_ids_json</code> on <code>ApprovalRequest</code> .
KG	Knowledge Graph — the semantic-search-indexed corpus of ingested documents, scoped by org.
Lean CI	Post-Sprint-4 CI model: <code>pytest</code> + <code>npm</code> build run locally before push; GitHub does only auto-merge of labelled PRs.
MCP	Model Context Protocol — the unified per-integration adapter spec. 14 concrete adapters + BYO marketplace.
Model Registry	The S4-M epic: <code>provider/entry/call-log/assignment/preference</code> tables across 26 provider types with auto-discovery, capability heuristic, health-check beat, and the resolver chain.

ModelEntry	One row per discovered (or manually added) model under a ModelProvider. Carries pricing, capability, context window, and feature flags.
ModelCallLog	Append-only audit log of every model call: token counts, latency, cost, request_id (joinable to AgentThread).
OpenAI-compatible	Provider type accepting any OpenAI-compatible endpoint (Ollama, OpenRouter, Together, Fireworks, Groq, DeepSeek). Drives the Conductor as of PR #370.
PLAN-v1.2	The roadmap document (PLAN-v1.2.md); the stakeholder reads this. The version line was renamed in Sprint 3 to align release tags with this document.
QuotaCounter	Sliding-window quota enforcement with a circuit-breaker; protects against runaway agent runs.
Reasoning-trace replay	GET /agents/runs/{request_id}/trace: chronological ModelCallLog rows for one agent run.
Resolver chain	model_resolver.py's precedence: user preference → org assignment → pool default → env override → stub fallback.
Slug	Human-readable identifier: users u-{first4}-{6hex}, orgs o-{first4}-{6hex}, bootstrap s-admin-000000.
Station	The supervised UI a human operator runs DClaw from. Approval Inbox lives here.
Trust mode	Per-action autonomy posture: hard_gate (always wait for approval), soft_gate (auto-execute, ask only on risk signals), auto (no gate, just audit).

**Five sprints in.
Four releases out.
One decision owed
before v1.2.0.**